

The Complete RAG & GenAI Mastery Book

Prepared for Tejas

The Complete RAG & GenAI Mastery Book	
How to Use This Book	
INTRODUCTION — What is RAG, Really?	
PHASE 0 — Foundations You Need Before Touching RAG	
0.1 Embeddings (the language of similarity)	
0.2 Cosine Similarity & Vector Math (just enough)	
0.3 Transformer/LLM Recap (just what's relevant to RAG)	
PHASE 1 — Naive RAG (The Core Loop)	
1.1 Document Loading	
1.2 Chunking Strategies	
1.3 Embedding Models — Choosing One	
1.4 Vector Databases	
1.5 Retriever — Similarity Search	
1.6 Generator — Combining Retrieval + LLM	
2.1 Hybrid Retrieval (Keyword + Vector Search)	
2.2 Reranking (Cross-Encoders)	
2.3 Query Transformation	
2.4 Parent-Child (Hierarchical) Retrieval	
2.5 Metadata Filtering	
2.6 RAG-Fusion	
3.1 Self-RAG	
3.2 Adaptive-RAG	
3.3 Agentic RAG (LangGraph)	
3.4 Corrective RAG (CRAG)	
4.1 Why Vector Search Fails on Relationship Questions	
4.2 Knowledge Graph Basics	
4.3 Building a Simple Knowledge Graph from Text	
4.4 Querying the Graph	
4.5 Hybrid: GraphRAG + VectorRAG Together	
5.1 The Full Routing Architecture	
5.2 Cost & Latency Optimization	
PHASE 6 — Evaluation (The Part Most Fresher Candidates Skip)	
6.1 Key Metrics	
6.2 Building a Golden Evaluation Set	
6.3 Automated Evaluation with RAGAS	
PHASE 7 — Trustworthiness & Deployment	
7.1 Hallucination Reduction Checklist	
7.2 Deployment — Wrapping Your RAG Pipeline as an API	
CAPSTONE PROJECT ROADMAP (Recommended Build Order)	
INTERVIEW-READY QUESTION BANK (Test Yourself Per Phase)	
END-TO-END SUMMARY TABLE	

The Complete RAG & GenAI Mastery Book

A Deep-Dive, Project-Based Guide (2026 Edition)

For: Tejas — Data Science aspirant, Pune **Goal:** Go from zero to production-level RAG understanding — theory, code, projects, and practice tasks in one place.

How to Use This Book

- Follow the phases **in order** — each builds on the previous one.
 - Every topic has: **Concept** → **Real-world analogy** → **Why it matters** → **Code** → **Practice Task**.
 - Don't skip the practice tasks. RAG is a *systems* skill — you learn it by building, breaking, and fixing pipelines, not by reading.
 - Treat each phase as a mini-project milestone. By the end of Phase 5, you'll have a portfolio-ready Agentic RAG system.
-

INTRODUCTION — What is RAG, Really?

The Core Problem

LLMs like GPT/Claude/Llama are frozen in time — they only know what they were trained on, and they hallucinate when asked about things outside that knowledge (private company docs, today's news, your own PDF files).

Real-world analogy: Imagine hiring a brilliant new employee who has read every book ever published up to 2025 — but has *never seen your company's internal documents* and can't look anything up. If you ask them a company-specific question, they'll either say "I don't know" or, worse, confidently make something up.

RAG solves this by giving the LLM a **librarian**: before answering, it fetches the relevant pages from your actual documents and hands them to the LLM along with the question. Now the LLM answers *using real, grounded information* instead of guessing from memory.

The Two Halves of Every RAG System

1. **Retriever** — finds the most relevant chunks of information for a given query.
2. **Generator** — the LLM that reads those chunks + the query and writes the final answer.

Everything you'll learn in this book is really just: *make the retriever smarter* and *make the generator use retrieved info better*.

PHASE 0 — Foundations You Need Before Touching RAG

0.1 Embeddings (the language of similarity)

Concept: An embedding is a list of numbers (a vector) that represents the *meaning* of a piece of text. Texts with similar meaning get vectors that are numerically close together.

Real-world analogy: Imagine plotting every word/sentence in a giant 3D space based on meaning. “King” and “Queen” would sit near each other. “King” and “Banana” would sit far apart. Embeddings just do this in much higher dimensions (usually 384 to 3072 numbers per piece of text).

Why it matters: RAG retrieval is fundamentally “find text with an embedding close to my question’s embedding.”

Code — generating your first embeddings:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2') # small, fast,
free, local

sentences = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "The stock market crashed today."
]

embeddings = model.encode(sentences)
print(embeddings.shape) # (3, 384) -> 3 sentences, 384 numbers each

# Check similarity
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity([embeddings[0]], [embeddings[1]])
print("Cat vs Feline similarity:", sim[0][0]) # High, ~0.7-0.8
sim2 = cosine_similarity([embeddings[0]], [embeddings[2]])
print("Cat vs Stock Market similarity:", sim2[0][0]) # Low, ~0.1
```

Practice Task 0.1: Take 10 sentences from any news article, embed them, and print a similarity matrix. Manually verify which pairs the model thinks are related — do you agree?

0.2 Cosine Similarity & Vector Math (just enough)

Concept: Cosine similarity measures the *angle* between two vectors, not their distance. Ranges from -1 (opposite) to 1 (identical direction).

Why it matters: It’s the default scoring function almost every vector database uses to rank retrieved documents.

```
import numpy as np

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

Practice Task 0.2: Implement cosine similarity from scratch (above) and verify it matches sklearn's output on the same vectors.

0.3 Transformer/LLM Recap (just what's relevant to RAG)

Concept you need: Context window (how much text the LLM can "see" at once) and tokens (chunks of text, roughly $\frac{3}{4}$ of a word each).

Why it matters for RAG: Your retrieved chunks + question + system prompt all have to fit inside the LLM's context window. This is *the* constraint that shapes chunking strategy (Phase 1.2).

Practice Task 0.3: Pick an LLM you'll use (e.g., Claude, GPT-4, Llama-3). Look up its context window size. Calculate how many 500-token chunks could fit alongside a 200-token question and a 300-token system prompt.

PHASE 1 — Naive RAG (The Core Loop)

This is the foundation. Everything else in this book is an upgrade to one of these 6 steps.

1.1 Document Loading

Concept: Before you can retrieve anything, you need to get your raw documents (PDFs, Word docs, web pages, CSVs) into plain text.

Real-world analogy: Think of this as unpacking boxes of files into readable pages before you can file them into a library.

Code:

```
from langchain_community.document_loaders import PyPDFLoader,
TextLoader

# Loading a PDF
loader = PyPDFLoader("company_handbook.pdf")
pages = loader.load()
print(f"Loaded {len(pages)} pages")
print(pages[0].page_content[:300]) # preview first page

# Loading plain text
loader2 = TextLoader("notes.txt")
docs = loader2.load()
```

Practice Task 1.1: Load 3 different file types (PDF, TXT, and a webpage using WebBaseLoader). Print the total character count of each.

1.2 Chunking Strategies

Concept: LLMs and vector search work better on small, focused pieces of text rather than entire documents. Chunking splits documents into bite-sized pieces.

Real-world analogy: Instead of handing someone an entire encyclopedia to find one fact, you tear out just the relevant paragraph. Chunking decides *how* you tear the pages — too small and you lose context, too big and you drown the LLM in irrelevant text.

Key strategies: - **Fixed-size chunking** — split every N characters/tokens (simple, sometimes cuts sentences awkwardly) - **Recursive chunking** — tries paragraph breaks first, then sentences, then words (the 2026 default — start with 300-500 tokens, 10-15% overlap) - **Semantic chunking** — splits based on meaning shifts, not fixed size (more advanced, covered in Phase 2)

Code:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,          # ~500 tokens per chunk
    chunk_overlap=75,        # 15% overlap to preserve context across
    boundaries
    separators=["\n\n", "\n", ". ", " ", ""]

    chunks = splitter.split_documents(pages)
    print(f"Split into {len(chunks)} chunks")
    print(chunks[0].page_content)
```

Why overlap matters (real example): If a sentence like “The refund policy applies only to items purchased within 30 days” gets split exactly at “30” — without overlap, one chunk says “...items purchased within” and the next says “30 days” — both useless alone. Overlap prevents this.

Practice Task 1.2: Take a 5-page PDF. Chunk it three ways — chunk_size=100, 500, and 1500. Manually inspect 3 chunks from each. Which size preserves meaning best? Which loses the most context?

1.3 Embedding Models — Choosing One

Concept: Not all embedding models are equal. Choice depends on cost, speed, and domain.

Model	Type	Best for
all-MiniLM-L6-v2	Local, free	Learning, prototyping
text-embedding-3-small (OpenAI)	API, cheap	Production, general text
bge-large-en	Local, free	High accuracy, needs GPU
Domain-tuned (e.g., legal-BERT)	Specialized	Medical/legal/finance docs

Practice Task 1.3: Embed the same 20 chunks with all-MiniLM-L6-v2 and bge-large-en. Compare retrieval results for 3 test questions — does the bigger model retrieve noticeably better chunks?

1.4 Vector Databases

Concept: A vector database stores embeddings and lets you search “find the top-k closest vectors to this query vector” *fast*, even across millions of documents.

Real-world analogy: A normal database finds exact matches (“find row where name = ‘Tejas’”). A vector database finds *similarity* matches (“find rows that mean something close to this”).

Start here: FAISS (Facebook AI Similarity Search) — free, runs locally, perfect for learning. **Move to for projects:** ChromaDB or Qdrant (easier persistence, filtering, production-ready).

Code — full mini pipeline with FAISS:

```
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-
v2")

vectorstore = FAISS.from_documents(chunks, embedding_model)

# Save it so you don't have to re-embed every time
vectorstore.save_local("faiss_index")

# Load it later
vectorstore = FAISS.load_local("faiss_index", embedding_model,
allow_dangerous_deserialization=True)
```

Practice Task 1.4: Build a FAISS index from a 20-page PDF. Save and reload it. Confirm retrieval still works after reload without re-embedding.

1.5 Retriever — Similarity Search

Concept: Given a query, convert it to an embedding, then fetch the top-k closest chunks from the vector store.

Code:

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

query = "What is the refund policy?"
results = retriever.invoke(query)

for i, doc in enumerate(results):
    print(f"--- Chunk {i+1} ---")
    print(doc.page_content[:200])
```

Common mistake (real-time gotcha): People set k too high (like k=20) thinking “more context = better answer.” In reality, this drowns the LLM in noise and increases hallucination risk. Start with k=3-5

and tune based on evaluation (Phase 6).

Practice Task 1.5: Test $k=1$, $k=3$, $k=8$ on the same question. Which gives the most accurate, focused answer when passed to an LLM?

1.6 Generator — Combining Retrieval + LLM

Concept: Build a prompt that includes the retrieved chunks as “context,” then asks the LLM to answer *only* using that context.

Code — Full Naive RAG Pipeline (end to end):

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

prompt_template = ChatPromptTemplate.from_template("""
You are a helpful assistant. Answer the question using ONLY the
context below.
If the answer isn't in the context, say "I don't have enough
information to answer that."

Context:
{context}

Question: {question}

Answer:
""")

def naive_rag(query):
    docs = retriever.invoke(query)
    context = "\n\n".join([d.page_content for d in docs])
    prompt = prompt_template.format(context=context, question=query)
    response = llm.invoke(prompt)
    return response.content

print(naive_rag("What is the refund policy?"))
```

Why the “say I don’t know” instruction matters: Without it, LLMs will confidently fabricate an answer even when the context has nothing relevant. This single line of prompt engineering is one of the highest-leverage hallucination fixes in all of RAG.

Practice Task 1.6 (MILESTONE PROJECT): Build a complete PDF Q&A chatbot end-to-end using everything in Phase 1. Use any PDF (your resume, a textbook chapter, a company policy doc). It should: load → chunk → embed → store → retrieve → generate. Wrap it in a simple loop so you can ask multiple questions.

(Continued in Part 2: Advanced RAG — hybrid search, reranking, query transformation) # PHASE 2 — Advanced RAG (Fixing Naive RAG’s Weaknesses)

Naive RAG fails in predictable ways: it misses keyword-specific matches (like exact product codes), retrieves loosely related but unhelpful chunks, and struggles with vague or complex questions. Phase 2 fixes each of these.

2.1 Hybrid Retrieval (Keyword + Vector Search)

The problem it solves: Vector search is great at *meaning* but bad at *exact terms*. If someone asks “What does error code E-4021 mean?”, a vector search might retrieve chunks about “error handling” in general — missing the chunk that literally contains “E-4021” — because embeddings don’t preserve exact strings well.

Real-world analogy: Vector search is like asking a friend “what was that movie about a guy stuck in time?” (they’ll guess Groundhog Day even without the title). Keyword search (BM25) is like Ctrl+F — exact string matching. Hybrid search uses both and merges the results.

Code:

```
from langchain.retrievers import BM25Retriever, EnsembleRetriever

# Keyword-based retriever
bm25_retriever = BM25Retriever.from_documents(chunks)
bm25_retriever.k = 4

# Vector-based retriever (from Phase 1)
vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# Combine both with weights
hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6] # tune based on your evaluation results
(Phase 6)
)

results = hybrid_retriever.invoke("What does error code E-4021
mean?")
```

Practice Task 2.1: Create a test set of 5 questions — 2 that rely on exact terms/codes, 2 conceptual questions, 1 mixed. Run each through vector-only, BM25-only, and hybrid retrieval. Record which method wins for each question type.

2.2 Reranking (Cross-Encoders)

The problem it solves: Vector search retrieves top-k *fast* but somewhat imprecisely (it’s an approximate ranking). A reranker takes those top-k candidates and re-scores them much more precisely — like a second, smarter pass.

Real-world analogy: Vector search is a fast first-round resume screener who skims 100 resumes down to 10 in seconds. The reranker is the senior recruiter who actually reads those 10 carefully and picks the best 3.

Code:


```

from sentence_transformers import CrossEncoder

reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

def rerank(query, docs, top_n=3):
    pairs = [[query, doc.page_content] for doc in docs]
    scores = reranker.predict(pairs)
    scored_docs = sorted(zip(docs, scores), key=lambda x: x[1],
reverse=True)
    return [doc for doc, score in scored_docs[:top_n]]

initial_results = hybrid_retriever.invoke("What is the refund policy
for damaged items?")
final_results = rerank("What is the refund policy for damaged
items?", initial_results, top_n=3)

```

Why this matters in production: Retrieval (k=20) → Rerank → Top 3 is now the 2026 standard pattern. You retrieve broadly and cheaply, then narrow precisely and expensively (rerankers cost more compute per document than vector search, so you only run them on a small shortlist).

Practice Task 2.2: Retrieve k=15 documents for a question, then rerank down to top 3. Compare the top 3 before and after reranking — did the order change? Did any irrelevant chunk get correctly demoted?

2.3 Query Transformation

2.3a Query Rewriting

Concept: Users ask messy, ambiguous, or incomplete questions. Rewrite the query into a clearer form before retrieval.

```

rewrite_prompt = ChatPromptTemplate.from_template(
    "Rewrite this question to be clearer and more specific for a
document search "
    "system. Only output the rewritten question.\n\nOriginal:
{question}"
)

def rewrite_query(question):
    return
llm.invoke(rewrite_prompt.format(question=question)).content

# Example
original = "what about the late thing"
rewritten = rewrite_query(original)
print(rewritten) # -> "What is the policy regarding late fees or
late submissions?"

```

2.3b HyDE (Hypothetical Document Embeddings)

Concept: Instead of embedding the raw question, ask the LLM to *write a hypothetical answer* first, then embed *that* — because a hypothetical answer's phrasing is closer to how the real answer is written in your documents than the question's phrasing is.

Real-world analogy: If you’re searching a legal database, searching with “can I get my money back” performs worse than searching with “the customer is entitled to a refund under Section 4.2” — even though both mean the same thing — because documents are written in the second style. HyDE bridges this gap.

```
hyde_prompt = ChatPromptTemplate.from_template(
    "Write a hypothetical, detailed paragraph that would answer this question, "
    "as if it were an excerpt from an official document:\n\n{question}"
)

def hyde_retrieve(question, k=4):
    hypothetical_doc = llm.invoke(hyde_prompt.format(question=question)).content
    return vectorstore.similarity_search(hypothetical_doc, k=k)
```

2.3c Multi-Query Retrieval

Concept: Generate 3-5 rephrasings of the same question, retrieve for each, and merge/deduplicate results — this catches relevant chunks that any single phrasing might miss.

```
from langchain.retrievers.multi_query import MultiQueryRetriever

multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(), llm=llm
)

results = multi_query_retriever.invoke("How do I cancel my subscription?")
```

Practice Task 2.3: Pick one genuinely ambiguous question from your document set. Run it through plain retrieval, HyDE, and multi-query retrieval. Compare which chunks each method surfaces.

2.4 Parent-Child (Hierarchical) Retrieval

The problem it solves: Small chunks retrieve precisely but lack surrounding context. Large chunks have context but retrieve imprecisely. Parent-child retrieval gets the best of both: search on small “child” chunks, but return the larger “parent” section to the LLM.

Real-world analogy: You search a book’s index (small, precise entries) but when you find the right entry, you read the whole page (parent), not just the indexed sentence.

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)

store = InMemoryStore()
parent_retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=store,
    child_splitter=child_splitter,
```

```

        parent_splitter=parent_splitter,
    )
    parent_retriever.add_documents(pages)

```

Practice Task 2.4: Compare answer quality for a context-dependent question (“What were the three conditions mentioned right before the refund clause?”) using plain chunking vs. parent-child retrieval.

2.5 Metadata Filtering

Concept: Attach metadata (date, source, category, author) to chunks at ingestion time, and filter retrieval by it — e.g., “only search documents from 2025 tagged ‘HR policy’.”

```

# When creating chunks, attach metadata
for chunk in chunks:
    chunk.metadata["category"] = "HR"
    chunk.metadata["year"] = 2025

# Filtered retrieval (syntax varies by vector DB – example for Chroma)
results = vectorstore.similarity_search(
    "leave policy",
    filter={"category": "HR", "year": 2025}
)

```

Real-world use case: A company chatbot with 5 years of policy documents — metadata filtering ensures the bot answers with the *current* policy, not an outdated 2021 version, even if the old one is more textually similar to the query.

Practice Task 2.5: Tag your chunk set with at least 2 metadata fields. Write a query that would give a wrong answer without filtering but a correct one with filtering — prove the difference.

2.6 RAG-Fusion

Concept: Combine multi-query retrieval with **Reciprocal Rank Fusion (RRF)** — a scoring method that rewards documents appearing near the top across *multiple* query variants, not just one.

```

def reciprocal_rank_fusion(results_lists, k=60):
    scores = {}
    for results in results_lists:
        for rank, doc in enumerate(results):
            doc_id = doc.page_content
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank + 1)

    ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    return ranked

```

Practice Task 2.6 (MILESTONE PROJECT): Upgrade your Phase 1 PDF Q&A bot into an “Advanced RAG” version: add hybrid search + reranking + query rewriting. Evaluate on 10 test questions before and after — document the improvement (this comparison is gold for your resume/portfolio).

(Continued in Part 3: Agentic & Reflective RAG) # PHASE 3 — Agentic & Reflective RAG (2025-2026 Frontier)

This is where RAG stops being a fixed pipeline and starts making *decisions*: should I retrieve at all? Is what I retrieved good enough? Should I search again with a different strategy?

3.1 Self-RAG

Concept: The model is trained (or prompted) to decide when retrieval is even necessary, then critiques both the retrieved passages and its own generated answer using special reflection steps. Self-RAG trains the LLM to decide when to retrieve, then critique both retrieved passages and its own generated output using special reflection tokens — this on-demand approach avoids unnecessary retrieval for simple queries and catches unsupported claims before they reach the user.

Real-world analogy: A good researcher doesn't Google every single sentence they write. If asked "What's 2+2?" they answer directly. If asked "What was our Q3 revenue?" they *know* they need to check the report first. Self-RAG teaches the model this same judgment — plus, after retrieving, it double-checks: "does this passage actually support what I'm about to say?"

Simplified prompt-based version (without fine-tuning, achievable today):

```
decide_prompt = ChatPromptTemplate.from_template(
    "Does answering this question require looking up external documents, "
    "or can it be answered from general knowledge/conversation context? "
    "Answer only 'RETRIEVE' or 'DIRECT'.\n\nQuestion: {question}"
)

critique_prompt = ChatPromptTemplate.from_template(
    "Context: {context}\n\nGenerated Answer: {answer}\n\n"
    "Is every claim in the answer directly supported by the context above? "
    "Answer 'SUPPORTED' or list the unsupported claims."
)

def self_rag(question):
    decision = llm.invoke(decide_prompt.format(question=question)).content.strip()
    if decision == "DIRECT":
        return llm.invoke(question).content

    docs = retriever.invoke(question)
    context = "\n\n".join([d.page_content for d in docs])
    answer = llm.invoke(prompt_template.format(context=context, question=question)).content

    critique = llm.invoke(critique_prompt.format(context=context, answer=answer)).content
    if "SUPPORTED" not in critique:
        return f"[Caution – possible unsupported claim]\n{answer}\n\nNote: {critique}"
    return answer
```

Practice Task 3.1: Test this on 5 questions — 2 that need no retrieval (“What’s 12 * 8?”), 3 that do. Confirm the router correctly skips retrieval for the simple ones.

3.2 Adaptive-RAG

Concept: A classifier routes each query to the *cheapest pipeline that can actually answer it correctly* — simple factual lookups go to naive RAG, medium questions go to advanced RAG (hybrid + rerank), and genuinely complex multi-step questions go to full agentic RAG.

The current 2026 best-practice decision tree looks like this: is the answer in a single document chunk? → Naive RAG (add reranker for precision). Does it need facts from 2-3 documents? → Advanced RAG (hybrid + rerank + query transform). Does it need to reason across many documents with relationships? → GraphRAG. Does it need multi-step reasoning? → Agentic RAG.

Code — a simple classifier router:

```
classify_prompt = ChatPromptTemplate.from_template("""
Classify this question's complexity for a retrieval system. Reply
with exactly one word:
- SIMPLE: answerable from a single document chunk
- MODERATE: needs facts combined from 2-3 chunks
- COMPLEX: needs multi-step reasoning or comparison across many
documents

Question: {question}
""")

def adaptive_rag(question):
    complexity =
llm.invoke(classify_prompt.format(question=question)).content.strip()

    if complexity == "SIMPLE":
        return naive_rag(question)                # from Phase 1
    elif complexity == "MODERATE":
        return advanced_rag(question)              # from Phase 2,
hybrid+rerank
    else:
        return agentic_rag(question)               # defined in 3.3
below
```

Practice Task 3.2: Write 6 test questions spanning all 3 complexity levels. Run them through the classifier — does it route correctly? Manually correct any misclassifications and note the pattern.

3.3 Agentic RAG (LangGraph)

Concept: Instead of one retrieval pass, the LLM acts as an *agent* — it can decide to search, look at results, decide they’re insufficient, reformulate, search again, use a calculator or other tool, and only then answer. This handles multi-hop questions naive RAG cannot.

Real-world analogy: Naive RAG is like asking a librarian a question and getting handed one book. Agentic RAG is like a research assistant who reads that book, realizes it references another source, goes and

finds *that* source too, cross-checks the numbers, and *then* comes back with a synthesized answer.

Code — simple agentic loop with LangGraph:

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class AgentState(TypedDict):
    question: str
    documents: List[str]
    answer: str
    iterations: int

def retrieve_node(state: AgentState):
    docs = retriever.invoke(state["question"])
    return {"documents": [d.page_content for d in docs],
"iterations": state["iterations"] + 1}

def grade_node(state: AgentState):
    context = "\n\n".join(state["documents"])
    grade_prompt = f"Do these documents contain enough info to
answer '{state['question']}'? Reply YES or NO.\n\n{context}"
    grade = llm.invoke(grade_prompt).content.strip()
    return {"documents": state["documents"]} if grade == "YES" or
state["iterations"] >= 3 else {"documents": []}

def rewrite_node(state: AgentState):
    new_q = rewrite_query(state["question"])
    return {"question": new_q}

def generate_node(state: AgentState):
    context = "\n\n".join(state["documents"])
    answer = llm.invoke(prompt_template.format(context=context,
question=state["question"])).content
    return {"answer": answer}

def should_continue(state: AgentState):
    return "generate" if state["documents"] or state["iterations"]
>= 3 else "rewrite"

graph = StateGraph(AgentState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("grade", grade_node)
graph.add_node("rewrite", rewrite_node)
graph.add_node("generate", generate_node)

graph.set_entry_point("retrieve")
graph.add_edge("retrieve", "grade")
graph.add_conditional_edges("grade", should_continue, {"generate":
"generate", "rewrite": "rewrite"})
graph.add_edge("rewrite", "retrieve")
graph.add_edge("generate", END)

agentic_rag_app = graph.compile()

result = agentic_rag_app.invoke({"question": "Compare the 2023 and
2024 refund policies", "documents": [], "answer": "", "iterations":
0})
print(result["answer"])
```

Practice Task 3.3 (MILESTONE PROJECT): Build this exact agentic loop on your document set. Deliberately test it with a question your naive RAG bot answered *wrong* in Phase 1 — confirm the agentic version retries and gets it right (or explain why it still fails, and iterate on the grading logic).

3.4 Corrective RAG (CRAG)

Concept: An extension of the grading idea above — if retrieved documents are graded as irrelevant/insufficient, the system falls back to a **web search** instead of just giving up or retrying the same vector store.

```
def corrective_rag(question):
    docs = retriever.invoke(question)
    context = "\n\n".join([d.page_content for d in docs])
    grade = llm.invoke(f"Are these documents relevant to
'{question}'? YES/NO\n\n{context}").content
    if "NO" in grade:
        # fallback: web search (pseudo-code – plug in your search
        tool)

        web_results = web_search_tool(question)
        context = web_results
    return llm.invoke(prompt_template.format(context=context,
question=question)).content
```

Practice Task 3.4: Ask your bot a question that’s *not* in your document set at all. Confirm plain RAG says “I don’t know” while CRAG correctly falls back to web search and answers.

(Continued in Part 4: GraphRAG) # PHASE 4 — GraphRAG
(Relationship-Aware Retrieval)

4.1 Why Vector Search Fails on Relationship Questions

The problem: Ask “Which vendors does our company share supply chain risk with, given the 2024 factory closures?” — this needs the model to trace relationships across many documents (vendor A supplies to B, B was affected by closure X, therefore...). Vector similarity search retrieves *topically similar* chunks, but can’t trace multi-hop *relationships* between entities scattered across different documents.

GraphRAG shines for relationship queries across large document collections — regulatory compliance analysis, research synthesis, competitive intelligence — while for simple factual retrieval, vector RAG remains faster, cheaper, and equally accurate. Don’t reach for GraphRAG by default; reach for it when your questions are fundamentally about *connections*.

4.2 Knowledge Graph Basics

Concept: A knowledge graph stores facts as **triples**: (Subject, Relationship, Object). E.g., (Tejas, learned, RAG), (RAG, uses, Vector Database).

Real-world analogy: A vector database is like a pile of index cards sorted by topic similarity. A knowledge graph is like a family tree or org chart — it explicitly draws the *lines* connecting entities, so you can traverse “who reports to whom” or “what depends on what” precisely.

4.3 Building a Simple Knowledge Graph from Text

Code — extracting triples with an LLM, storing in NetworkX:

```
import networkx as nx
import json

extract_prompt = ChatPromptTemplate.from_template("""
Extract entity relationships from this text as a JSON list of
triples:
[{"subject": "...", "relation": "...", "object": "..."}]
Only output valid JSON.

Text: {text}
""")

def extract_triples(text):
    response = llm.invoke(extract_prompt.format(text=text)).content
    return json.loads(response)

graph = nx.DiGraph()

for chunk in chunks[:20]: # start small, extraction is LLM-call-
    expensive
    triples = extract_triples(chunk.page_content)
    for t in triples:
        graph.add_edge(t["subject"], t["object"],
            relation=t["relation"])

    print(f"Graph has {graph.number_of_nodes()} entities and
{graph.number_of_edges()} relationships")
```

4.4 Querying the Graph

Code — multi-hop traversal for relationship questions:

```
def graph_query(entity, max_hops=2):
    """Find everything connected to an entity within max_hops"""
    if entity not in graph:
        return "Entity not found in graph."

    subgraph_nodes = nx.single_source_shortest_path_length(graph,
entity, cutoff=max_hops)
    facts = []
    for node in subgraph_nodes:
        for neighbor in graph.successors(node):
```



```

        relation = graph[node][neighbor]["relation"]
        facts.append(f"{node} --{relation}--> {neighbor}")
    return "\n".join(facts)

    context = graph_query("Vendor A", max_hops=2)
    answer = llm.invoke(prompt_template.format(context=context,
question="What risks is Vendor A connected to?")).content

```

4.5 Hybrid: GraphRAG + VectorRAG Together

Real production pattern: Use the graph to find *which entities/documents are relevant* via relationships, then use vector search *within* that narrowed set for the actual detailed text passages. This combines relationship reasoning with precise text retrieval.

```

def graph_plus_vector_rag(question, entity):
    graph_context = graph_query(entity, max_hops=2)
    vector_docs = vectorstore.similarity_search(question, k=3)
    vector_context = "\n\n".join([d.page_content for d in
vector_docs])
    combined_context = f"Relationships:\n{graph_context}\n\nDetailed
text:\n{vector_context}"
    return
llm.invoke(prompt_template.format(context=combined_context,
question=question)).content

```

Practice Task 4.1 (MILESTONE PROJECT): Pick a small document set with clear relationships (e.g., a company org chart doc, a set of research papers that cite each other, or a supply chain description). Build a knowledge graph from it and answer a genuinely multi-hop question that plain vector RAG gets wrong. Document the before/after difference — this is a strong differentiator for interviews since most fresher candidates never touch GraphRAG.

Note on tooling: For serious projects, look into **Neo4j** (graph database) + **LangChain's GraphRAG integrations**, or Microsoft's open-source **GraphRAG** library, once you're comfortable with the NetworkX version above.

(Continued in Part 5: Adaptive/Production Routing, Part 6: Evaluation, Part 7: Deployment) # PHASE 5 — Adaptive/Production RAG Systems

5.1 The Full Routing Architecture

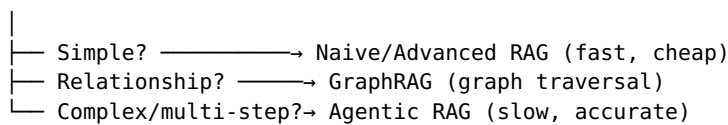
RAG in 2026 isn't a single technique — it's a spectrum, and the best production systems use adaptive routing to match query complexity to pipeline complexity: keeping costs low for simple questions while deploying the full agentic or graph-based arsenal only for queries that genuinely need it.

The full decision flow:

```

User Query
|
▼
Complexity Classifier

```



Code — putting Phases 1-4 together into one production router:

```
def production_rag(question):
    # Step 1: classify
    classification = llm.invoke(f"""
    Classify this question. Reply with exactly one word:
    - SIMPLE (single fact lookup)
    - RELATIONSHIP (needs tracing connections between entities)
    - COMPLEX (needs multi-step reasoning or comparison)
    Question: {question}
    """).content.strip()

    # Step 2: route
    if classification == "SIMPLE":
        return naive_rag(question)
    elif classification == "RELATIONSHIP":
        entity = llm.invoke(f"Extract the main entity from:
        {question}").content.strip()
        return graph_plus_vector_rag(question, entity)
    else:
        return agentic_rag_app.invoke(
            {"question": question, "documents": [], "answer": "",
            "iterations": 0}
        )["answer"]
```

Practice Task 5.1: Build this router on top of everything you built in Phases 1-4. Test with 9 questions — 3 of each type — and confirm correct routing.

5.2 Cost & Latency Optimization

Key techniques: - **Caching** — cache embeddings for repeated queries; cache full answers for FAQ-like questions - **Batching** — embed documents in batches, not one at a time - **Tiered models** — use a cheap/fast LLM for classification and query rewriting, save the expensive model for final generation

```
import hashlib
answer_cache = {}

def cached_rag(question):
    key = hashlib.md5(question.lower().strip().encode()).hexdigest()
    if key in answer_cache:
        return answer_cache[key]
    answer = production_rag(question)
    answer_cache[key] = answer
    return answer
```

Practice Task 5.2: Time your pipeline with and without caching on a repeated question. Note the latency difference.

PHASE 6 — Evaluation (The Part

Most Fresher Candidates Skip)

6.1 Key Metrics

- **Faithfulness** — is the answer actually supported by the retrieved context (not hallucinated)?
- **Answer Relevance** — does the answer actually address the question asked?
- **Context Precision** — of the retrieved chunks, how many were actually relevant?
- **Context Recall** — did retrieval find *all* the relevant information, or miss some?

Real-world analogy: Faithfulness = “did the student only use the textbook, not make things up?” Context recall = “did the student find *all* the relevant textbook pages, or miss some?” These are separate failure modes and need separate fixes.

6.2 Building a Golden Evaluation Set

Concept: A small, hand-verified set of (question, expected answer, expected source chunks) triples — your ground truth for testing every pipeline change.

```
golden_set = [  
    {  
        "question": "What is the refund window?",  
        "expected_answer": "30 days from purchase",  
        "expected_source": "Section 4.2"  
    },  
    # ... 15-20 more, covering easy/medium/hard questions  
]
```

6.3 Automated Evaluation with RAGAS

```
from ragas import evaluate  
from ragas.metrics import faithfulness, answer_relevancy,  
context_precision, context_recall  
from datasets import Dataset  
  
eval_data = {  
    "question": [item["question"] for item in golden_set],  
    "answer": [production_rag(item["question"]) for item in  
golden_set],  
    "contexts": [[d.page_content for d in  
retriever.invoke(item["question"])] for item in golden_set],  
    "ground_truth": [item["expected_answer"] for item in  
golden_set],  
}  
  
dataset = Dataset.from_dict(eval_data)  
results = evaluate(dataset, metrics=[faithfulness, answer_relevancy,  
context_precision, context_recall])  
print(results)
```

Practice Task 6.1 (MILESTONE PROJECT): Build a 15-question golden evaluation set for your project. Run RAGAS on your Naive RAG (Phase 1) vs. your Advanced/Agentic RAG (Phase 2-3). Produce a before/after comparison table — this single artifact is one of the strongest things you can show in a data science interview, because it proves you understand *why* your improvements actually worked, not just that you copied more code.

PHASE 7 — Trustworthiness & Deployment

7.1 Hallucination Reduction Checklist

- Always instruct the LLM to say “I don’t know” when context is insufficient (Phase 1.6)
- Use faithfulness checks (Self-RAG style, Phase 3.1) before returning answers
- Cite sources — return which chunk/document supported each claim

Code — adding citations:

```
def rag_with_citations(question):
    docs = retriever.invoke(question)
    context = "\n\n".join([f"[Source {i+1}]: {d.page_content}" for
i, d in enumerate(docs)])
    cited_prompt = f"""Answer using the sources below. After each
claim, cite the source number like [Source 1].

{context}

Question: {question}"""
    return llm.invoke(cited_prompt).content
```

7.2 Deployment — Wrapping Your RAG Pipeline as an API

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Query(BaseModel):
    question: str

@app.post("/ask")
def ask(query: Query):
    answer = production_rag(query.question)
    return {"question": query.question, "answer": answer}

# Run with: uvicorn main:app --reload
```

Dockerfile (basic):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Practice Task 7.1 (FINAL MILESTONE PROJECT): Wrap your best-performing RAG pipeline (from Phase 5) as a FastAPI service, containerize it with Docker, and test it with sample requests via curl or Postman. This is now a deployable, portfolio-ready project.

CAPSTONE PROJECT ROADMAP (Recommended Build Order)

#	Project	Phases Used	Resume Value
1	PDF Q&A Bot	Phase 1	Foundation
2	Advanced Q&A Bot (hybrid + rerank) with before/after evaluation	Phase 2 + 6	Shows depth
3	Agentic Research Assistant (multi-hop, self-correcting)	Phase 3	Shows 2026 skills — maps to your existing Multi-Agent Research System
4	Relationship-aware document analyzer (e.g., supply chain, org chart, citation network)	Phase 4	Rare, strong differentiator
5	Deployed, evaluated, production RAG API	Phase 5 + 7	Full-stack proof

Suggested application to your existing projects: - **Medical Chatbot** → rebuild it through Phase 2 (hybrid+rerank is critical for medical accuracy) then add Phase 6 evaluation — faithfulness matters enormously in medical contexts. - **Multi-Agent Research System** → this is already conceptually Phase 3/5 territory. Formalize it using the LangGraph pattern in 3.3 and add the adaptive router from 5.1.

INTERVIEW-READY QUESTION BANK (Test Yourself Per Phase)

1. Why does chunk overlap matter? Give a concrete example of what breaks without it.
2. When would you choose BM25 over vector search? When would hybrid beat both alone?
3. Why rerank instead of just retrieving more documents (k=20) directly?
4. Explain HyDE in one sentence, and why it works.
5. What's the difference between Self-RAG and Corrective RAG?
6. When is GraphRAG worth the added complexity vs. vector RAG?
7. What's context precision vs. context recall, and how would you improve each independently?
8. How would you reduce hallucination in a RAG system without retraining the LLM?
9. Design an adaptive routing system for a customer support bot — what would your 3 tiers be?
10. What's the cost/latency tradeoff of adding a reranker to a production pipeline?

Practice Task (ongoing): After finishing each phase, answer the corresponding questions above out loud, as if in an interview, without looking at your notes. If you stumble, that's your signal to revisit that section.

END-TO-END SUMMARY TABLE

Phase	Topic	Core Skill Gain
0	Embeddings, cosine similarity, context windows	Foundation intuition
1	Naive RAG (load→chunk→embed→store→retrieve→generate)	Build a working RAG pipeline
2	Hybrid search, reranking, query transformation, parent-child, metadata filtering, RAG-Fusion	Make retrieval precise
3	Self-RAG, Adaptive-RAG, Agentic RAG, Corrective RAG	Make the system reason and correct
4	Knowledge graphs, GraphRAG	Handle relationship hop questions
5	Adaptive routing, caching, cost optimization	Build production-grade system
6	Faithfulness, relevance, precision/recall, RAGAS	Prove your system actually works
7	Citations, hallucination reduction, FastAPI, Docker	Ship it

You now have a complete, sequenced, code-backed path from zero to a deployable, evaluated, agentic RAG system. Build one milestone project per phase — don't rush past the practice tasks, since they're what will actually show up in interview questions and real debugging scenarios.